

Cross-Platform Cryptography in .NET

09/07/2024

Cryptographic operations in .NET are done by operating system (OS) libraries. This dependency has advantages:

- .NET apps benefit from OS reliability. Keeping cryptography libraries safe from vulnerabilities is a high priority for OS vendors. To do that, they provide updates that system administrators should be applying.
- .NET apps have access to FIPS-validated algorithms if the OS libraries are FIPS-validated.

The dependency on OS libraries also means that .NET apps can only use cryptographic features that the OS supports. While all platforms support certain core features, some features that .NET supports can't be used on some platforms. This article identifies the features that are supported on each platform.

This article assumes you have a working familiarity with cryptography in .NET. For more information, see [.NET Cryptography Model](#) and [.NET Cryptographic Services](#).

Hash and Message Authentication Algorithms

All hash algorithm and hash-based message authentication (HMAC) classes, including the *Managed classes, defer to the OS libraries with the exception of .NET on Browser WASM. In Browser WASM, SHA-1, SHA-2-256, SHA-2-384, SHA-2-512 and the HMAC equivalents are implemented using managed code.

 Expand table

Algorithm	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android	Browser
MD5	✓	✓	✓	✓	✓	✗
SHA-1	✓	✓	✓	✓	✓	✓
SHA-2-256	✓	✓	✓	✓	✓	✓
SHA-2-384	✓	✓	✓	✓	✓	✓
SHA-2-512	✓	✓	✓	✓	✓	✓

Algorithm	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android	Browser
SHA-3-256	Windows 11 Build 25324+	OpenSSL 1.1.1+	✗	✗	✗	✗
SHA-3-384	Windows 11 Build 25324+	OpenSSL 1.1.1+	✗	✗	✗	✗
SHA-3-512	Windows 11 Build 25324+	OpenSSL 1.1.1+	✗	✗	✗	✗
SHAKE-128	Windows 11 Build 25324+	OpenSSL 1.1.1+ ²	✗	✗	✗	✗
SHAKE-256	Windows 11 Build 25324+	OpenSSL 1.1.1+ ²	✗	✗	✗	✗
HMAC-MD5	✓	✓	✓	✓	✓	✗
HMAC-SHA-1	✓	✓	✓	✓	✓	✓
HMAC-SHA-2-256	✓	✓	✓	✓	✓	✓
HMAC-SHA-2-384	✓	✓	✓	✓	✓	✓
HMAC-SHA-2-512	✓	✓	✓	✓	✓	✓
HMAC-SHA-3-256	Windows 11 Build 25324+	OpenSSL 1.1.1+	✗	✗	✗	✗
HMAC-SHA-3-384	Windows 11 Build 25324+	OpenSSL 1.1.1+	✗	✗	✗	✗
HMAC-SHA-3-512	Windows 11 Build 25324+	OpenSSL 1.1.1+	✗	✗	✗	✗
KMAC-128 ¹	Windows 11 Build 26016+	OpenSSL 3.0+	✗	✗	✗	✗
KMAC-256 ¹	Windows 11 Build 26016+	OpenSSL 3.0+	✗	✗	✗	✗
KMAC-XOF-128 ¹	Windows 11 Build 26016+	OpenSSL 3.0+	✗	✗	✗	✗

Algorithm	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android	Browser
KMAC-XOF-256 ¹	Windows 11 Build 26016+	OpenSSL 3.0+	✗	✗	✗	✗

¹Available starting in .NET 9.

²Streaming extensible output function (XOF) is available starting in .NET 9. On Linux, this requires OpenSSL 3.3.

Symmetric encryption

The underlying ciphers and chaining are done by the system libraries.

 Expand table

Cipher + Mode	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
AES-CBC	✓	✓	✓	✓	✓
AES-ECB	✓	✓	✓	✓	✓
AES-CFB8	✓	✓	✓	✓	✓
AES-CFB128	✓	✓	✓	✓	✓
3DES-CBC	✓	✓	✓	✓	✓
3DES-ECB	✓	✓	✓	✓	✓
3DES-CFB8	✓	✓	✓	✓	✓
3DES-CFB64	✓	✓	✓	✓	✓
DES-CBC	✓	✓	✓	✓	✓
DES-ECB	✓	✓	✓	✓	✓
DES-CFB8	✓	✓	✓	✓	✓
RC2-CBC	✓	✓	✓	✓	✗
RC2-ECB	✓	✓	✓	✓	✗
RC2-CFB	✗	✗	✗	✗	✗

Authenticated encryption

Authenticated encryption (AE) support is provided for AES-CCM, AES-GCM, and ChaCha20Poly1305 via the [System.Security.Cryptography.AesCcm](#), [System.Security.Cryptography.AesGcm](#), and [System.Security.Cryptography.ChaCha20Poly1305](#) classes, respectively.

Since authenticated encryption requires newer platform APIs to support the algorithm, support may not be present on all platforms. The `IsSupported` static property on the classes for the algorithm can be used to detect at runtime if the current platform supports the algorithm or not.

 Expand table

Cipher + Mode	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android	Browser
AES-GCM	✓	✓	✓	⚠	✓	✗
AES-CCM	✓	✓	⚠	✗	✓	✗
ChaCha20Poly1305	Windows 10 Build 20142+	OpenSSL 1.1.0+	✓	⚠	API Level 28+	✗

AES-CCM on macOS

On macOS, the system libraries don't support AES-CCM for third-party code, so the [AesCcm](#) class uses OpenSSL for support. Users on macOS need to obtain an appropriate copy of OpenSSL (libcrypto) for this type to function, and it must be in a path that the system would load a library from by default. We recommend that you install OpenSSL from a package manager such as Homebrew.

The `libcrypto.0.9.7.dylib` and `libcrypto.0.9.8.dylib` libraries included in macOS are from earlier versions of OpenSSL and will not be used. The `libcrypto.35.dylib`, `libcrypto.41.dylib`, and `libcrypto.42.dylib` libraries are from LibreSSL and will not be used.

AES-GCM and ChaCha20Poly1305 on iOS, tvOS, and MacCatalyst

Support for AES-GCM and ChaCha20Poly1305 is available starting in .NET 9 on iOS and tvOS 13.0 and later, and all versions of MacCatalyst.

AES-CCM keys, nonces, and tags

- Key Sizes

AES-CCM works with 128, 192, and 256-bit keys.

- Nonce Sizes

The [AesCcm](#) class supports 56, 64, 72, 80, 88, 96, and 104-bit (7, 8, 9, 10, 11, 12, and 13-byte) nonces.

- Tag Sizes

The [AesCcm](#) class supports creating or processing 32, 48, 64, 80, 96, 112, and 128-bit (4, 8, 10, 12, 14, and 16-byte) tags.

AES-GCM keys, nonces, and tags

- Key Sizes

AES-GCM works with 128, 192, and 256-bit keys.

- Nonce Sizes

The [AesGcm](#) class supports only 96-bit (12-byte) nonces.

- Tag Sizes On Windows and Linux, the [AesGcm](#) class supports creating or processing 96, 104, 112, 120, and 128-bit (12, 13, 14, 15, and 16-byte) tags. On Apple platforms, the tag size is limited to 128-bit (16-byte) due to limitations of the CryptoKit framework.

ChaCha20Poly1305 keys, nonces, and tags.

ChaCha20Poly1305 has a fixed size for the key, nonce, and authentication tag. ChaCha20Poly1305 always uses a 256-bit key, a 96-bit (12-byte) nonce, and 128-bit (16-byte) tag.

Asymmetric cryptography

This section includes the following subsections:

- [RSA](#)
- [ECDSA](#)

- [ECDH](#)
- [DSA](#)

RSA

RSA (Rivest–Shamir–Adleman) key generation is performed by the OS libraries and is subject to their size limitations and performance characteristics.

RSA key operations are performed by the OS libraries, and the types of key that can be loaded are subject to OS requirements.

.NET does not expose "raw" (unpadded) RSA operations.

Padding and digest support vary by platform:

[Expand table](#)

Padding Mode	Windows (CNG)	Linux (OpenSSL)	macOS	iOS, tvOS, MacCatalyst	Android	Windows (CAPI)
PKCS1 Encryption	✓	✓	✓	✓	✓	✓
OAEP - SHA-1	✓	✓	✓	✓	✓	✓
OAEP - SHA-2	✓	✓	✓	✓	✓	✗
OAEP - SHA-3	Windows 11 Build 25324+	OpenSSL 1.1.1+	✗	✗	✗	✗
PKCS1 Signature (MD5, SHA-1)	✓	✓	✓	✓	✓	✓
PKCS1 Signature (SHA-2)	✓	✓	✓	✓	✓	⚠ ¹
PKCS1 Signature (SHA-3)	Windows 11 Build 25324+	OpenSSL 1.1.1+	✗	✗	✗	✗
PSS	✓	✓	✓	✓	✓	✗

¹ Windows CryptoAPI (CAPI) is capable of PKCS1 signature with a SHA-2 algorithm. But the individual RSA object may be loaded in a cryptographic service provider (CSP) that doesn't support it.

RSA on Windows

- Windows CryptoAPI (CAPI) is used whenever [new RSACryptoServiceProvider\(\)](#) is used.
- Windows Cryptography API Next Generation (CNG) is used whenever [new RSACng\(\)](#) is used.
- The object returned by [RSA.Create](#) is internally powered by Windows CNG. This use of Windows CNG is an implementation detail and is subject to change.
- The [GetRSAPublicKey](#) extension method for [X509Certificate2](#) returns an [RSACng](#) instance. This use of [RSACng](#) is an implementation detail and is subject to change.
- The [GetRSAPrivateKey](#) extension method for [X509Certificate2](#) currently prefers an [RSACng](#) instance, but if [RSACng](#) can't open the key, [RSACryptoServiceProvider](#) will be attempted. The preferred provider is an implementation detail and is subject to change.

RSA native interop

.NET exposes types to allow programs to interoperate with the OS libraries that the .NET cryptography code uses. The types involved do not translate between platforms, and should only be directly used when necessary.

[Expand table](#)

Type	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
RSACryptoServiceProvider	✓	⚠ ¹	⚠ ¹	⚠ ¹	⚠ ¹
RSACng	✓	✗	✗	✗	✗
RSAOpenSsl	✗	✓	⚠ ²	✗	✗

¹ On non-Windows, [RSACryptoServiceProvider](#) can be used for compatibility with existing programs. In that case, any method that requires OS interop, such as opening a named key, throws a [PlatformNotSupportedException](#).

² On macOS, [RSAOpenSsl](#) works if OpenSSL is installed and an appropriate libcrypto dylib can be found via dynamic library loading. If an appropriate library can't be found, exceptions will be thrown.

ECDSA

ECDSA (Elliptic Curve Digital Signature Algorithm) key generation is done by the OS libraries and is subject to their size limitations and performance characteristics.

ECDSA key curves are defined by the OS libraries and are subject to their limitations.

 Expand table

Elliptic Curve	Windows 10	Windows 7 - 8.1	Linux	macOS	iOS, tvOS, MacCatalyst	Android
NIST P-256 (secp256r1)	✓	✓	✓	✓	✓	✓
NIST P-384 (secp384r1)	✓	✓	✓	✓	✓	✓
NIST P-521 (secp521r1)	✓	✓	✓	✓	✓	✓
Brainpool curves (as named curves)	✓	✗	⚠ ¹	✗	✗	⚠ ⁴
Other named curves	⚠ ²	✗	⚠ ¹	✗	✗	⚠ ⁴
Explicit curves	✓	✗	✓	✗	✗	✓
Export or import as explicit	✓	✗ ³	✓	✗ ³	✗ ³	✓

¹ Linux distributions don't all have support for the same named curves.

² Support for named curves was added to Windows CNG in Windows 10. For more information, see [CNG Named Elliptic Curves](#). Named curves are not available in earlier versions of Windows, except for three curves in Windows 7.

³ Exporting with explicit curve parameters requires OS library support, which is not available on Apple platforms or earlier versions of Windows.

⁴ Android support for some curves depends on the Android version. Android distributors may choose to add or remove curves from their Android build as well.

ECDSA Native Interop

.NET exposes types to allow programs to interoperate with the OS libraries that the .NET cryptography code uses. The types involved don't translate between platforms and should only be directly used when necessary.

 Expand table

Type	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
ECDsaCng	✓	✗	✗	✗	✗
ECDsaOpenSsl	✗	✓	⚠ *	✗	✗

* On macOS, [ECDsaOpenSsl](#) works if OpenSSL is installed in the system and an appropriate libcrypto dylib can be found via dynamic library loading. If an appropriate library can't be found, exceptions will be thrown.

ECDH

ECDH (Elliptic Curve Diffie-Hellman) key generation is done by the OS libraries and is subject to their size limitations and performance characteristics.

The [ECDiffieHellman](#) class supports the "raw" value of the ECDH computation as well as through the following key derivation functions:

- HASH(Z)
- HASH(prepend || Z || append)
- HMAC(key, Z)
- HMAC(key, prepend || Z || append)
- HMAC(Z, Z)
- HMAC(Z, prepend || Z || append)
- Tls11Prf(label, seed)

ECDH key curves are defined by the OS libraries and are subject to their limitations.

[Expand table](#)

Elliptic Curve	Windows 10	Windows 7 - 8.1	Linux	macOS	iOS, tvOS, MacCatalyst	Android
NIST P-256 (secp256r1)	✓	✓	✓	✓	✓	✓
NIST P-384 (secp384r1)	✓	✓	✓	✓	✓	✓
NIST P-521 (secp521r1)	✓	✓	✓	✓	✓	✓
Brainpool curves (as named curves)	✓	✗	⚠ ¹	✗	✗	⚠ ⁴
Other named curves	⚠ ²	✗	⚠ ¹	✗	✗	⚠ ⁴

Elliptic Curve	Windows 10	Windows 7 - 8.1	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Explicit curves	✓	✗	✓	✗	✗	✓
Export or import as explicit	✓	✗ ³	✓	✗ ³	✗ ³	✓

¹ Linux distributions don't all have support for the same named curves.

² Support for named curves was added to Windows CNG in Windows 10. For more information, see [CNG Named Elliptic Curves](#). Named curves are not available in earlier versions of Windows, except for three curves in Windows 7.

³ Exporting with explicit curve parameters requires OS library support, which is not available on Apple platforms or earlier versions of Windows.

⁴ Android support for some curves depends on the Android version. Android distributors may choose to add or remove curves from their Android build as well.

ECDH native interop

.NET exposes types to allow programs to interoperate with the OS libraries that .NET uses. The types involved don't translate between platforms and should only be directly used when necessary.

[Expand table](#)

Type	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
ECDiffieHellmanCng	✓	✗	✗	✗	✗
ECDiffieHellmanOpenSsl	✗	✓	⚠ [*]	✗	✗

^{*} On macOS, [ECDiffieHellmanOpenSsl](#) works if OpenSSL is installed and an appropriate libcrypto dylib can be found via dynamic library loading. If an appropriate library can't be found, exceptions will be thrown.

DSA

DSA (Digital Signature Algorithm) key generation is performed by the system libraries and is subject to their size limitations and performance characteristics.

 Expand table

Function	Windows CNG	Linux	macOS	Windows CAPI	iOS, tvOS, MacCatalyst	Android
Key creation (<= 1024 bits)	✓	✓	✗	✓	✗	✓
Key creation (> 1024 bits)	✓	✓	✗	✗	✗	✓
Loading keys (<= 1024 bits)	✓	✓	✓	✓	✗	✓
Loading keys (> 1024 bits)	✓	✓	⚠ *	✗	✗	✓
FIPS 186-2	✓	✓	✓	✓	✗	✓
FIPS 186-3 (SHA-2 signatures)	✓	✓	✗	✗	✗	✓

* macOS loads DSA keys bigger than 1024 bits, but the behavior of those keys is undefined. They don't behave according to FIPS 186-3.

DSA on Windows

- Windows CryptoAPI (CAPI) is used whenever [new DSACryptoServiceProvider\(\)](#) is used.
- Windows Cryptography API Next Generation (CNG) is used whenever [new DSACng\(\)](#) is used.
- The object returned by [DSA.Create](#) is internally powered by Windows CNG. This use of Windows CNG is an implementation detail and is subject to change.
- The [GetDSAPublicKey](#) extension method for [X509Certificate2](#) returns a [DSACng](#) instance. This use of [DSACng](#) is an implementation detail and is subject to change.
- The [GetDSAPrivateKey](#) extension method for [X509Certificate2](#) prefers an [DSACng](#) instance, but if [DSACng](#) can't open the key, [DSACryptoServiceProvider](#) will be attempted. The preferred provider is an implementation detail and is subject to change.

DSA native interop

.NET exposes types to allow programs to interoperate with the OS libraries that the .NET cryptography code uses. The types involved don't translate between platforms and should only be directly used when necessary.

 Expand table

Type	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
DSACryptoServiceProvider	✓	⚠ ¹	⚠ ¹	✗	⚠ ¹
DSACng	✓	✗	✗	✗	✗
DSAOpenSsl	✗	✓	⚠ ²	✗	✗

¹ On non-Windows, [DSACryptoServiceProvider](#) can be used for compatibility with existing programs. In that case, any method that requires system interop, such as opening a named key, throws a [PlatformNotSupportedException](#).

² On macOS, [DSAOpenSsl](#) works if OpenSSL is installed and an appropriate libcrypto dylib can be found via dynamic library loading. If an appropriate library can't be found, exceptions will be thrown.

X.509 Certificates

The majority of support for X.509 certificates in .NET comes from OS libraries. To load a certificate into an [X509Certificate2](#) or [X509Certificate](#) instance in .NET, the certificate must be loaded by the underlying OS library.

Read a PKCS12/PFX

 Expand table

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Empty	✓	✓	✓	✓	✓
One certificate, no private key	✓	✓	✓	✓	✓
One certificate, with private key	✓	✓	✓	✓	✓
Multiple certificates, no private keys	✓	✓	✓	✓	✓

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Multiple certificates, one private key	✓	✓	✓	✓	✓
Multiple certificates, multiple private keys	✓	✓	✓	✓	✓

Write a PKCS12/PFX

 Expand table

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Empty	✓	✓	✓	✓	✓
One certificate, no private key	✓	✓	✓	✓	✓
One certificate, with private key	✓	✓	✓	✓	✓
Multiple certificates, no private keys	✓	✓	✓	✓	✓
Multiple certificates, one private key	✓	✓	✓	✓	✓
Multiple certificates, multiple private keys	✓	✓	✓	✓	✓
Ephemeral loading	✓	✓	✗	✓	✓

macOS can't load certificate private keys without a keychain object, which requires writing to disk. Keychains are created automatically for PFX loading, and are deleted when no longer in use. Since the [X509KeyStorageFlags.EphemeralKeySet](#) option means that the private key should not be written to disk, asserting that flag on macOS results in a [PlatformNotSupportedException](#).

Write a PKCS7 certificate collection

Windows and Linux both emit DER-encoded PKCS7 blobs. macOS emits indefinite-length-CER-encoded PKCS7 blobs.

X509Store

On Windows, the [X509Store](#) class is a representation of the Windows Certificate Store APIs. Those APIs work the same in .NET Core and .NET 5 as they do in .NET Framework.

On non-Windows, the [X509Store](#) class is a projection of system trust decisions (read-only), user trust decisions (read-write), and user key storage (read-write).

The following tables show which scenarios are supported in each platform. For unsupported scenarios (✗ in the tables), a [CryptographicException](#) is thrown.

The My store

[Expand table](#)

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Open CurrentUser\My (ReadOnly)	✓	✓	✓	✓	✓
Open CurrentUser\My (ReadWrite)	✓	✓	✓	✓	✓
Open CurrentUser\My (ExistingOnly)	✓	⚠	✓	✓	✓
Open LocalMachine\My	✓	✗	✓	✓	✓

On Linux, stores are created on first write, and no user stores exist by default, so opening CurrentUser\My with ExistingOnly may fail.

On macOS, the CurrentUser\My store is the user's default keychain, which is login.keychain by default. The LocalMachine\My store is System.keychain.

The Root store

[Expand table](#)

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Open CurrentUser\Root (ReadOnly)	✓	✓	✓	✗	✓
Open CurrentUser\Root (ReadWrite)	✓	✓	✗	✗	✗

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Open CurrentUser\Root (ExistingOnly)	✓	⚠	✓ (if ReadOnly)	✗	✓ (if ReadOnly)
Open LocalMachine\Root (ReadOnly)	✓	✓	✓	✗	✓
Open LocalMachine\Root (ReadWrite)	✓	✗	✗	✗	✗
Open LocalMachine\Root (ExistingOnly)	✓	⚠	✓ (if ReadOnly)	✗	✓ (if ReadOnly)

On Linux, the LocalMachine\Root store is an interpretation of the CA bundle in the default path for OpenSSL.

On macOS, the currentUser\Root store is an interpretation of the SecTrustSettings results for the user trust domain. The LocalMachine\Root store is an interpretation of the SecTrustSettings results for the admin and system trust domains.

The Intermediate store

[Expand table](#)

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Open CurrentUser\Intermediate (ReadOnly)	✓	✓	✓	✗	✗
Open CurrentUser\Intermediate (ReadWrite)	✓	✓	✗	✗	✗
Open CurrentUser\Intermediate (ExistingOnly)	✓	⚠	✓ (if ReadOnly)	✗	✗
Open LocalMachine\Intermediate (ReadOnly)	✓	✓	✓	✗	✗
Open LocalMachine\Intermediate (ReadWrite)	✓	✗	✗	✗	✗

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Open LocalMachine\Intermediate (ExistingOnly)	✓	⚠	✓ (if ReadOnly)	✗	✗

On Linux, the `CurrentUser\Intermediate` store is used as a cache when downloading intermediate CAs by their Authority Information Access records on successful X509Chain builds. The `LocalMachine\Intermediate` store is an interpretation of the CA bundle in the default path for OpenSSL.

On macOS, the `CurrentUser\Intermediate` store is treated as a custom store. Certificates added to this store do not affect X.509 chain building.

The Disallowed store

[Expand table](#)

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Open CurrentUser\Disallowed (ReadOnly)	✓	⚠	✓	✓	✓
Open CurrentUser\Disallowed (ReadWrite)	✓	⚠	✗	✗	✗
Open CurrentUser\Disallowed (ExistingOnly)	✓	⚠	✓ (if ReadOnly)	✓ (if ReadOnly)	✓ (if ReadOnly)
Open LocalMachine\Disallowed (ReadOnly)	✓	✗	✓	✓	✓
Open LocalMachine\Disallowed (ReadWrite)	✓	✗	✗	✗	✗
Open LocalMachine\Disallowed (ExistingOnly)	✓	✗	✓ (if ReadOnly)	✓ (if ReadOnly)	✓ (if ReadOnly)

On Linux, the `Disallowed` store is not used in chain building, and attempting to add contents to it results in a [CryptographicException](#). A [CryptographicException](#) is thrown when opening the `Disallowed` store if it has already acquired contents.

On macOS, the `CurrentUser\Disallowed` and `LocalMachine\Disallowed` stores are interpretations of the appropriate `SecTrustSettings` results for certificates whose trust is set to `Always Deny`.

Nonexistent store

[Expand table](#)

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Open non-existent store (ExistingOnly)	✗	✗	✗	✗	✗
Open <code>CurrentUser</code> non-existent store (ReadWrite)	✓	✓	⚠	✗	✗
Open <code>LocalMachine</code> non-existent store (ReadWrite)	✓	✗	✗	✗	✗

On macOS, custom store creation with the `X509Store` API is supported only for `CurrentUser` location. It will create a new keychain with no password in the user's keychain directory (`~/Library/Keychains`). To create a keychain with password, a `P/Invoke` to `SecKeychainCreate` could be used. Similarly, `SecKeychainOpen` could be used to open keychains in different locations. The resulting `IntPtr` can be passed to [new X509Store\(IntPtr\)](#) to obtain a read/write-capable store, subject to the current user's permissions.

X509Chain

macOS doesn't support Offline CRL utilization, so `X509RevocationMode.Offline` is treated as `X509RevocationMode.Online`.

macOS doesn't support a user-initiated timeout on CRL (Certificate Revocation List) / OCSP (Online Certificate Status Protocol) / AIA (Authority Information Access) downloading, so `X509ChainPolicy.UrlRetrievalTimeout` is ignored.

Additional resources

- [.NET Cryptography Model](#)
- [.NET Cryptographic Services](#)
- [Timing vulnerabilities with CBC-mode symmetric decryption using padding](#)

- [ASP.NET Core Data Protection](#)